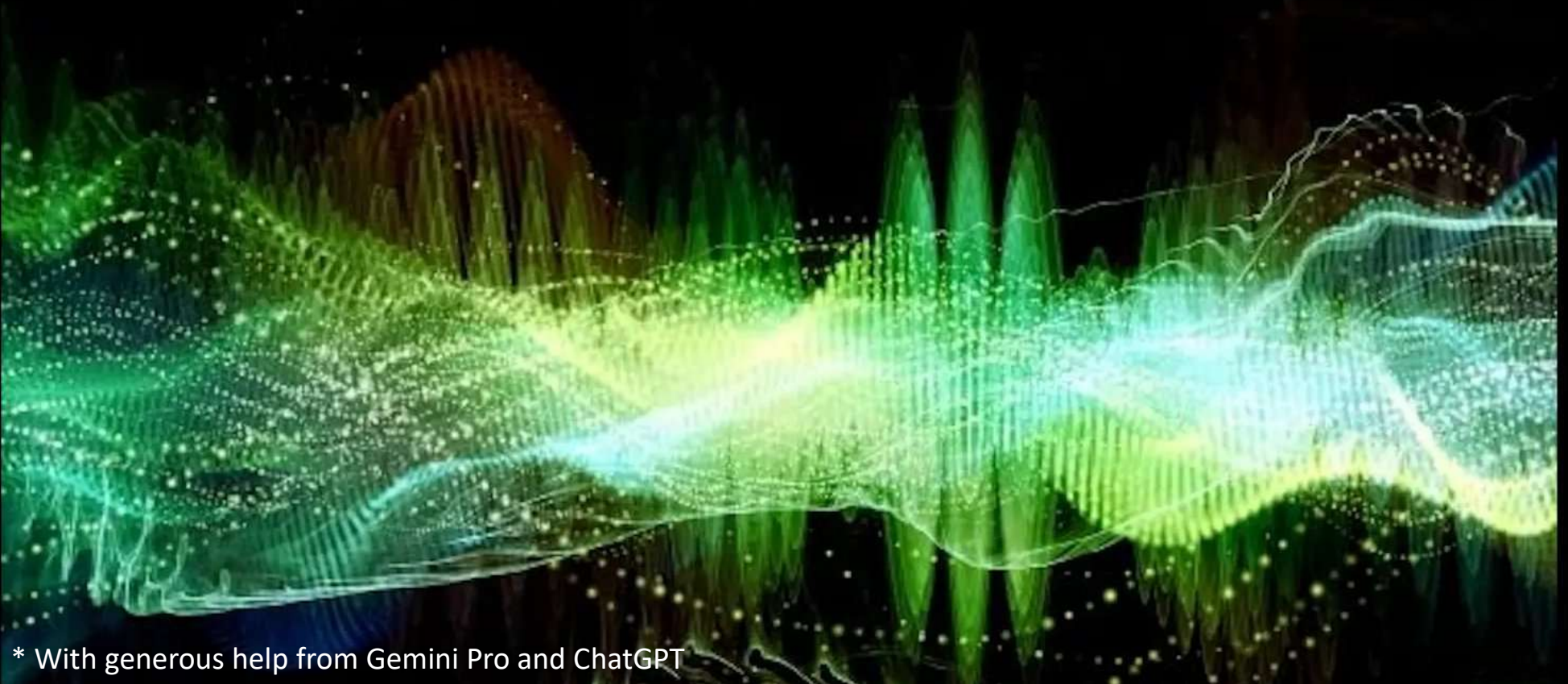


ECEN 4/5632 Theory and Application of Digital Filters

Lecture 7, 9-15-26

Peter Mathys*



* With generous help from Gemini Pro and ChatGPT

FIR Filter Design, Windowing, & 1D Convolutions

From ideal frequency responses to fixed or learnable convolution kernels



Where this 75-minute lecture is going

0–10 min • FIR vs. IIR

Linear phase, group delay, and why waveform alignment can matter.

10–30 min • Ideal filter + Gibbs

Derive the sinc impulse response and see what truncation does.

30–55 min • Window method

Main-lobe width versus sidelobe attenuation.

55–75 min • DSP ↔

PyTorch

Implement fixed FIR taps with Conv1d; then ask what changes when weights are learned.

DSP bridge: a mathematically designed FIR and a neural 1-D convolution use the same sliding weighted sum.

Classical DSP supplies structure and constraints; machine learning either reuses that structure or learns parameters inside it.

Same frequency-selective goal, different structures

FIR: feedforward

Finite impulse response. No feedback. Exact linear phase is available when the taps have the required symmetry.

IIR: recursive

Feedback permits sharp magnitude responses with fewer coefficients, but phase is generally nonlinear.

ML perspective

A fixed FIR can be the front end of an ML pipeline; a Conv1d layer uses the same local weighted sum but can make the coefficients trainable.

$$y[n] = \sum_{k=0}^M h[k]x[n-k]$$

Do not oversell linear phase: it is valuable when temporal alignment and waveform shape should be preserved, but not every ML task requires it.

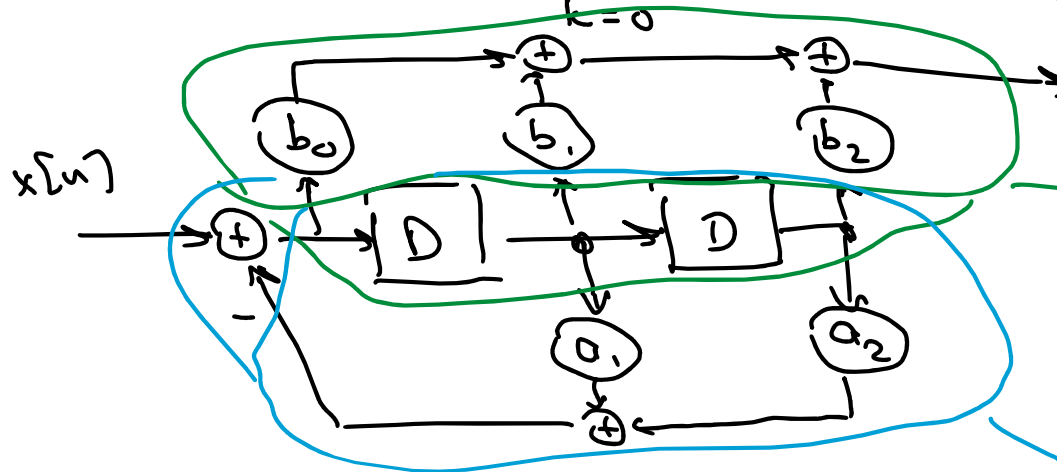
IIR :

$$y[n] = \sum_{k=0}^M b_k x[n-k] - \sum_{k=1}^N a_k \cdot y[n-k]$$

FIR filter depends only on $x[n-k]$
for $k=0, 1, 2, \dots$
for FIR filter
 $h[k] = b_k$

Structure of rational filter functions

difference eq
$$\sum_{k=0}^N b_k x[n-k] = \sum_{k=0}^N a_k y[n-k]$$



FIR part Lecture 7

General 2nd order rational filter function

IIR part Lecture 8

$$H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}}$$

Symmetric FIR taps give a constant delay

example
N=7

$\frac{N-1}{2}$: midpoint

$$h[n] = h[N-1-n]$$

all the phase is in here



$$H(e^{j\omega}) = e^{-j\omega(N-1)/2} A(\omega)$$

DTFT of $h[n]$ starting at $n=0$

DTFT of symmetric version of $h[n]$

$$\tau_g(\omega) = -\frac{d}{d\omega} \angle H(e^{j\omega}) = \frac{N-1}{2}$$

Physical interpretation

Every passband frequency component is delayed by the same number of samples, apart from sign-related phase jumps where the real amplitude term changes sign.

ML connection

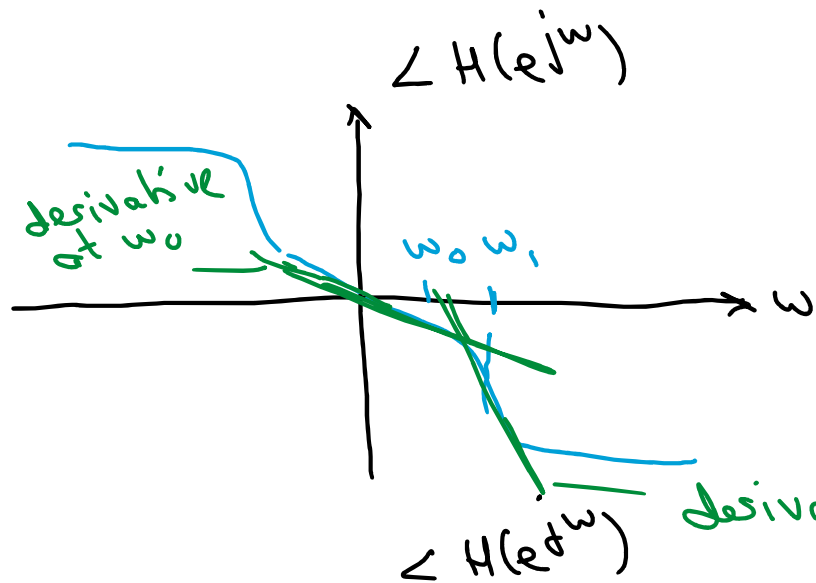
If a transient shape is a feature, constant delay avoids frequency-dependent smearing before MFCCs, peaks, onsets, or time-localized descriptors are computed.

phase $-\omega \frac{N-1}{2}$

$$\tau_g(\omega) = -\frac{d}{d\omega} [\text{phase}] = \frac{N-1}{2}$$

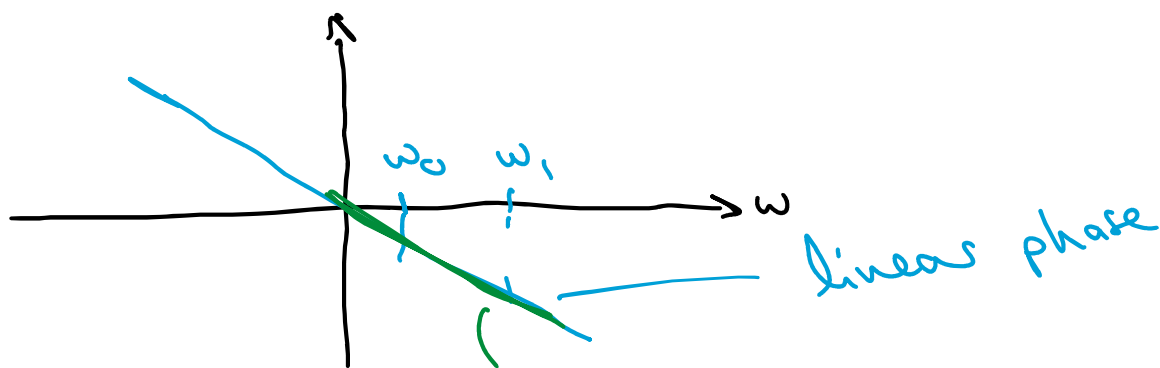
group delay

$$x[n-n_0] \Leftrightarrow X(e^{j\omega}) e^{-j\omega n_0}$$



non-linear as
varies with ω

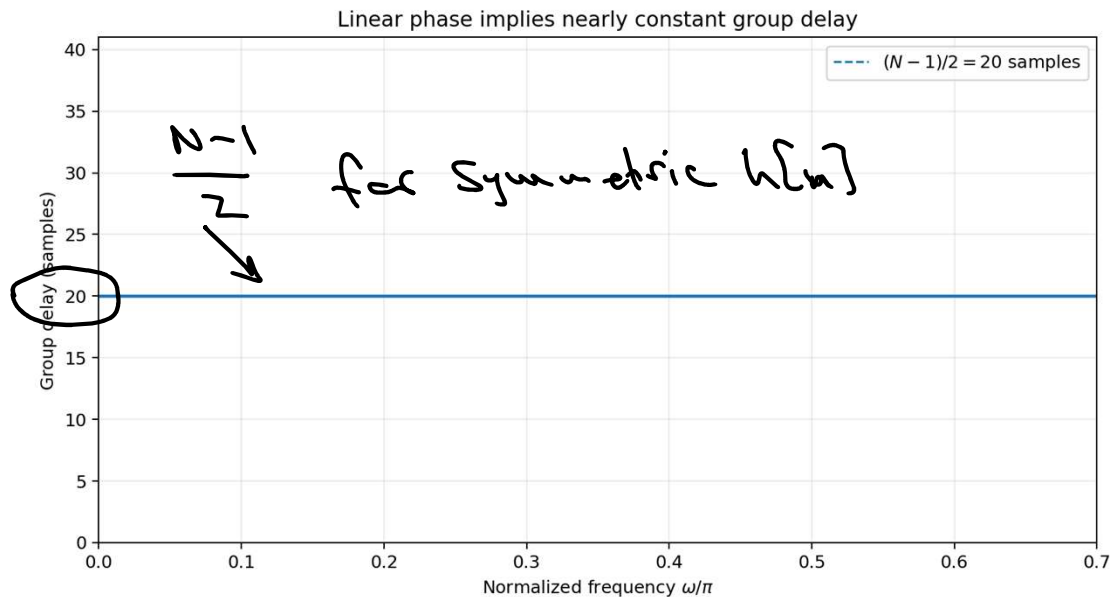
ω_1 would be delayed
by more than ω_0



same
derivative of phase
at ω_0 and ω_1

$\Rightarrow$ both frequencies
are delayed by
same amount

Group delay is the more intuitive diagnostic



Read the plot

For a 41-tap symmetric FIR, the expected delay is

$$\tau_g = \frac{41 - 1}{2} = 20 \text{ samples}$$

$$N = 41$$

Good answer

“Linear phase does not mean zero delay. It means a delay that is approximately constant with frequency.”

$$\tau_g = - \frac{d \angle H(e^{j\omega})}{d\omega}$$

CONCEPT CHECK

What would nonlinear phase do to a transient?

Question

Two filters have nearly identical magnitude responses. One has constant group delay; the other has strongly frequency-dependent group delay. What can differ at the output?

non-linear phase

linear phase

Good answer

The spectra can have almost the same magnitudes while the waveform shapes differ because frequency components arrive with different delays.

Strong answer

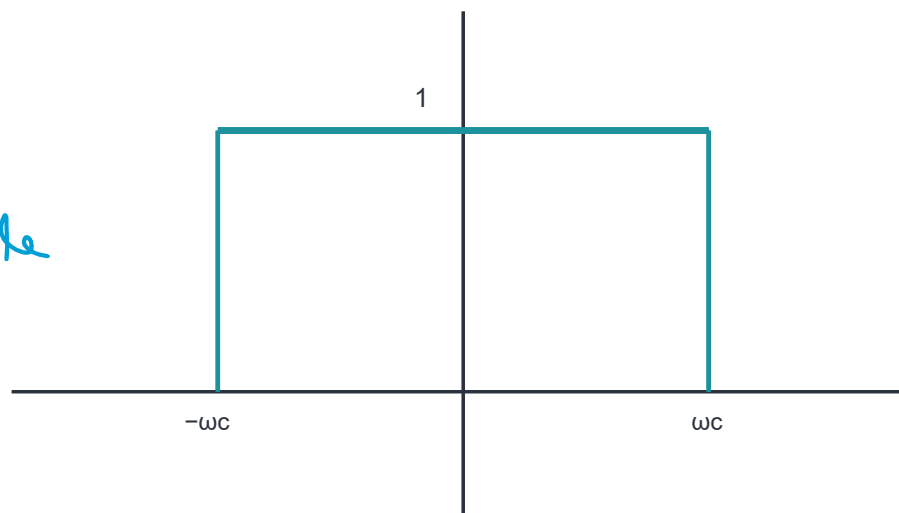
That distortion can alter time-localized features such as onset shape or peak timing. Whether it harms an ML model depends on what features and invariances the model uses.

Start in the frequency domain

DTFT

$$H_d(e^{j\omega}) = \begin{cases} 1, & |\omega| \leq \omega_c \\ 0, & \omega_c < |\omega| \leq \pi \end{cases}$$

repeated every multiple of 2π



Engineering dream

Perfectly flat passband, zero stopband, and an infinitely sharp transition.

Reality

The price is an infinite-duration, noncausal impulse response.

Inverse DTFT produces the sinc impulse response

$$h_d[n] = \frac{1}{2\pi} \int_{-\omega_c}^{\omega_c} e^{j\omega n} d\omega$$

$$\underline{h_d[n]} = \frac{e^{j\omega_c n} - e^{-j\omega_c n}}{2\pi j n} = \frac{\sin(\omega_c n)}{\underline{\pi n}}, \quad n \neq 0$$

$$h_d[0] = \lim_{n \rightarrow 0} \frac{\sin(\omega_c n)}{\pi n} = \frac{\omega_c}{\pi}$$

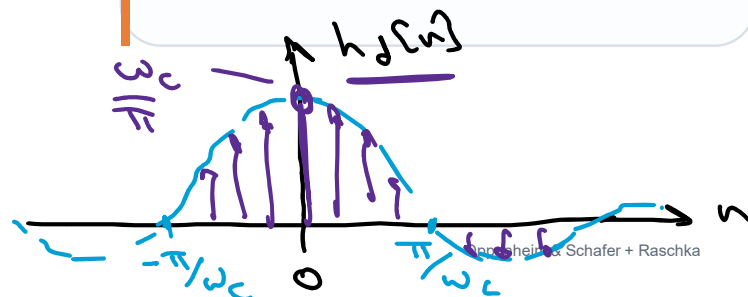
$$\underline{h_d[n]} = \frac{\omega_c}{\omega_c} \frac{\sin(\omega_c n)}{\pi n} = \underline{\frac{\omega_c}{\pi} \cdot \text{sinc}\left(\frac{\omega_c n}{\pi}\right)}$$

Interpretation

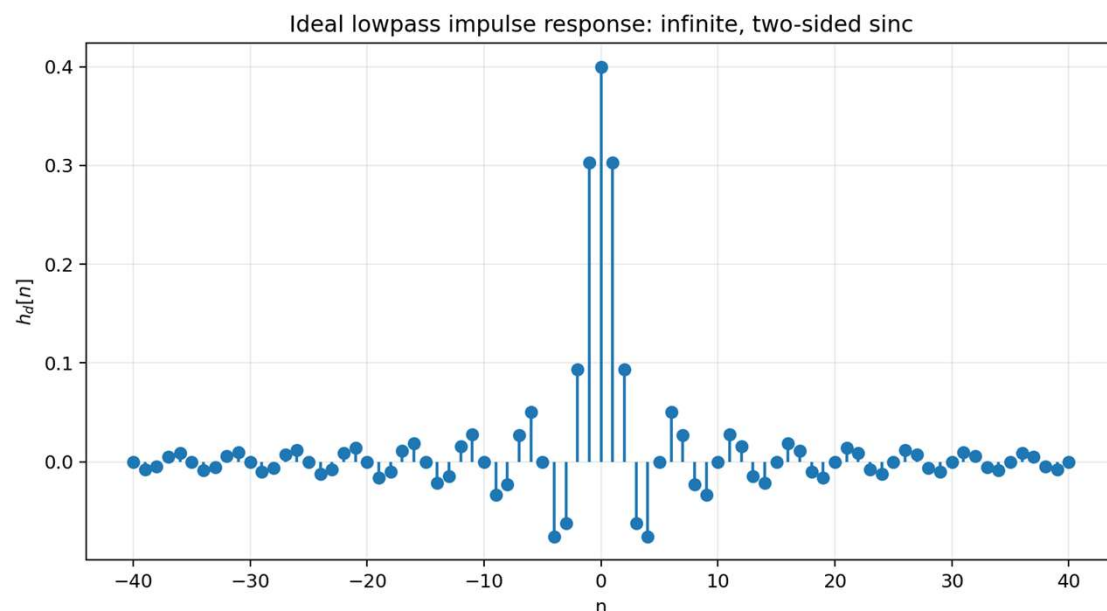
A brick-wall frequency response requires a sinc extending forever in both time directions.

Senior/grad expectation

Students should be able to derive the sinc from the inverse DTFT, not only quote the result.



The ideal impulse response is infinite and two-sided



non
causal



truncate and shift to make it causal

Problem 1: infinite length

A finite processor cannot store or multiply by infinitely many taps.

Problem 2: noncausal

Samples for negative time imply dependence on future inputs unless we delay/shift the finite approximation.

What we will do

Truncate the sinc, shift it to be causal, and shape the truncation with a window.

Center first, then shift to a causal tap sequence

$$h_d[n] = \frac{\sin(\omega_c(n - M/2))}{\pi(n - M/2)}$$

For an FIR of order M with $M+1$ taps, centering the ideal response at $M/2$ gives the symmetry needed for linear phase.

$$\underline{\tau_g} = \frac{M}{2} = \underline{\frac{N-1}{2}}$$

Example

For $N = 51$ taps, $M = 50$ and the delay is 25 samples.

Important

The shift changes phase, not magnitude. We are making the finite approximation causal by accepting a fixed delay.

$$\frac{M}{2} = \frac{N-1}{2} \quad \text{middle of response } h[n]$$

Rectangular truncation is simple—but its sidelobes are not

$$h_R[n] = h_d[n]w_R[n]$$

Time-domain action

The rectangular window keeps a finite block of sinc samples and sets everything else to zero.

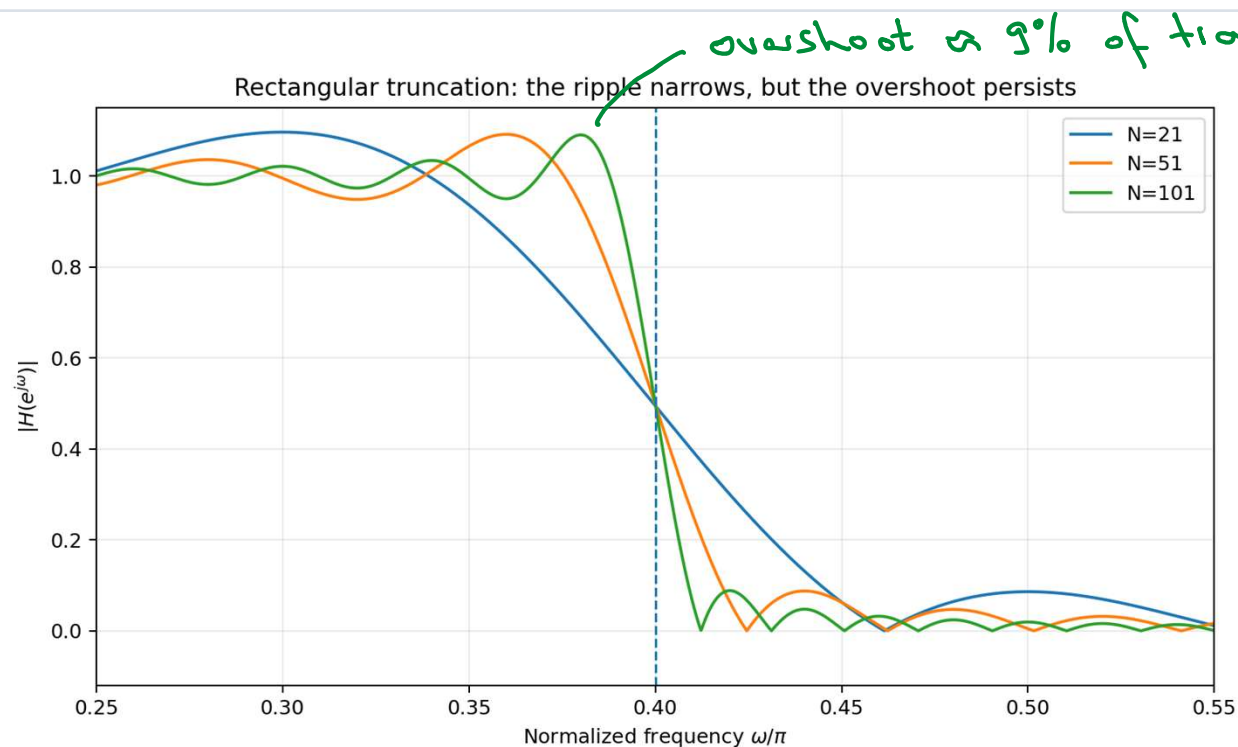
Frequency-domain consequence

Abrupt truncation has strong sidelobes. Convolution with the rectangular-window spectrum creates oscillatory passband/stopband ripple.

A common misconception: “make the filter longer and the Gibbs ripple disappears.”

A longer rectangular window makes the ripple region narrower, but the limiting peak overshoot near the discontinuity does not go to zero.

Increasing length narrows the transition but does not eliminate overshoot



Read the plot

As N increases, the oscillation is compressed toward the cutoff.

But notice

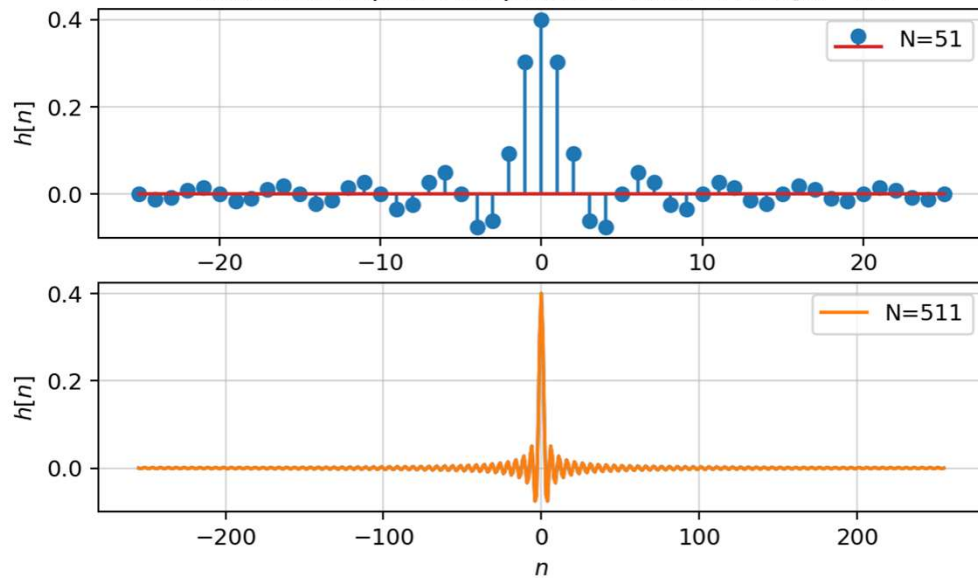
The peak overshoot remains visible. Length and sidelobe level are different design knobs.

Next step

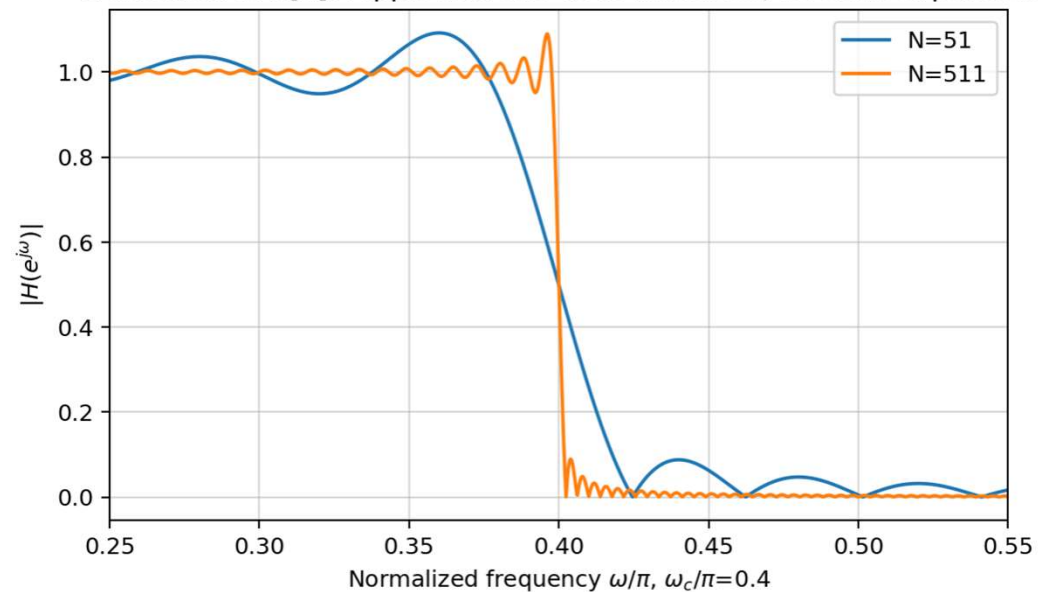
Use a smoother window to reduce sidelobes—accepting a wider transition.

Gibbs' Phenomenon

Truncated Impulse Response of Ideal LPF, $\omega_c/\pi=0.4$



Truncation of $h[n]$: Ripple narrows as N increases, Overshoot persists



Shape the truncation instead of switching it off abruptly

$$h_w[n] = h_d[n] \underbrace{w[n]}_{\text{window function}}$$

$$H_w(e^{j\omega}) = \frac{1}{2\pi} H_d(e^{j\omega}) * W(e^{j\omega})$$

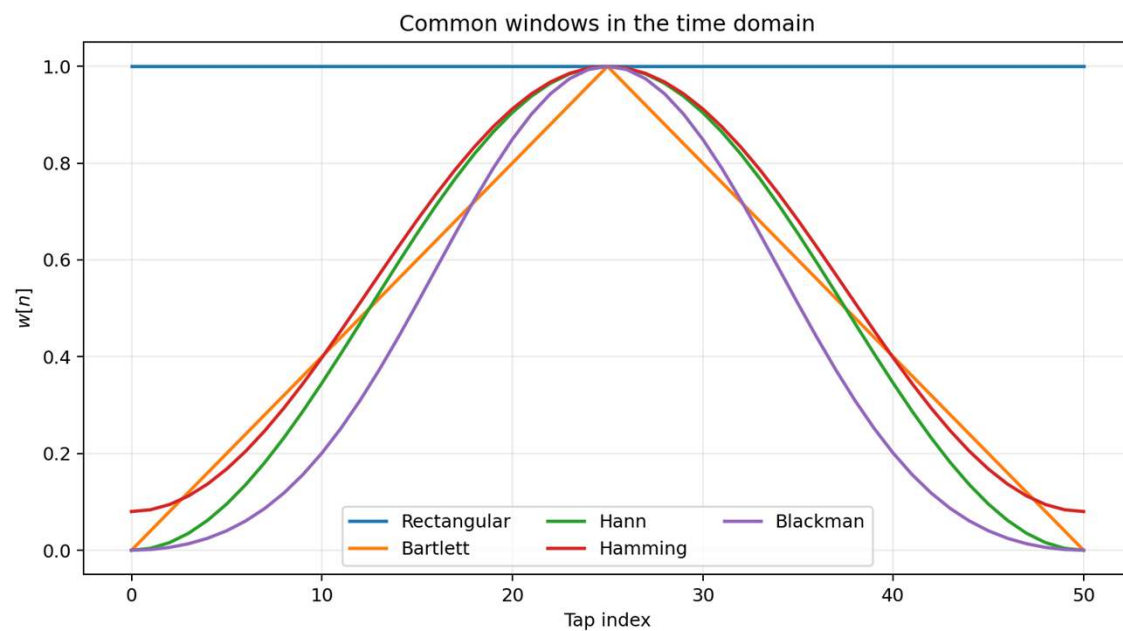
Key duality

Multiplication in time corresponds to periodic convolution in frequency.

Design intuition

A smoother time-domain taper lowers spectral sidelobes but broadens the window main lobe, which broadens the filter transition band.

Rectangular → Bartlett → Hann/Hamming → Blackman



Rectangular

Hardest edges; narrowest main lobe; high sidelobes.

Hann / Hamming

Smooth taper. Hamming is designed for a particularly low first sidelobe.

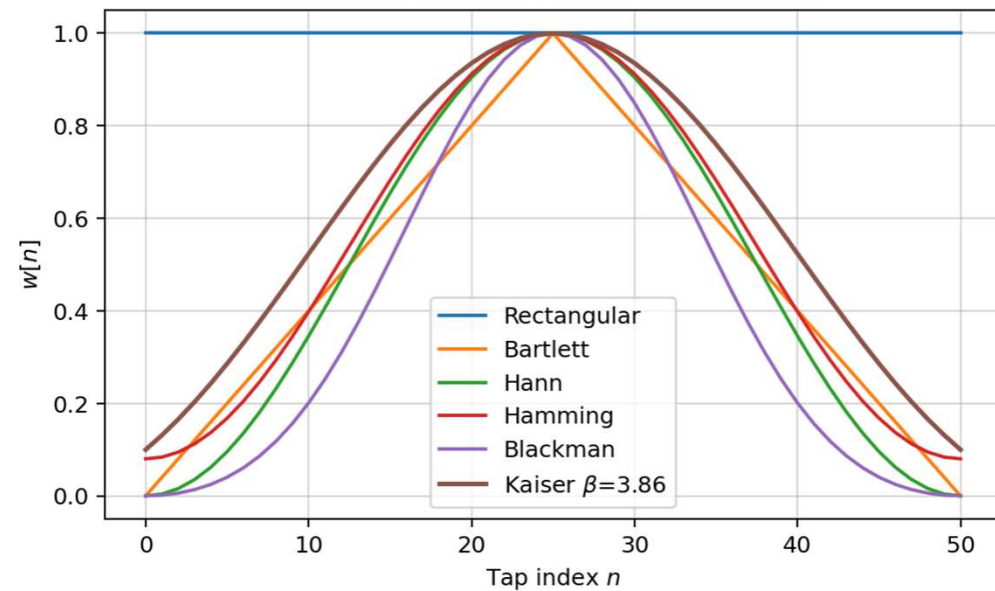
Blackman

Stronger taper and lower sidelobes, but a wider main lobe / transition.

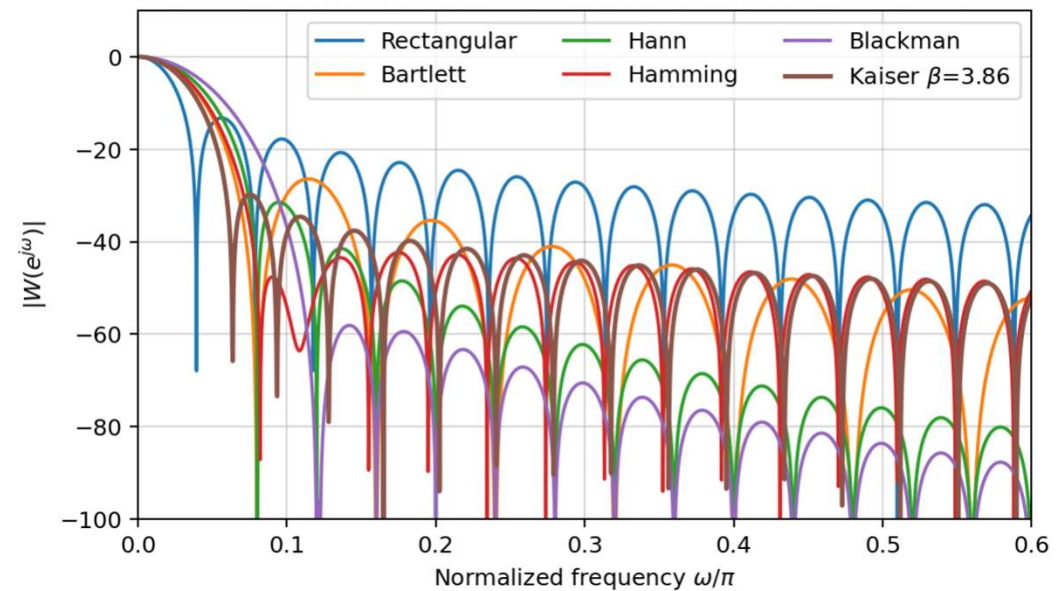
"Hann" is the standard name; "Hanning" is common informal usage.

Window Functions in Time and Frequency, N=51

Common Windows in the Time Domain

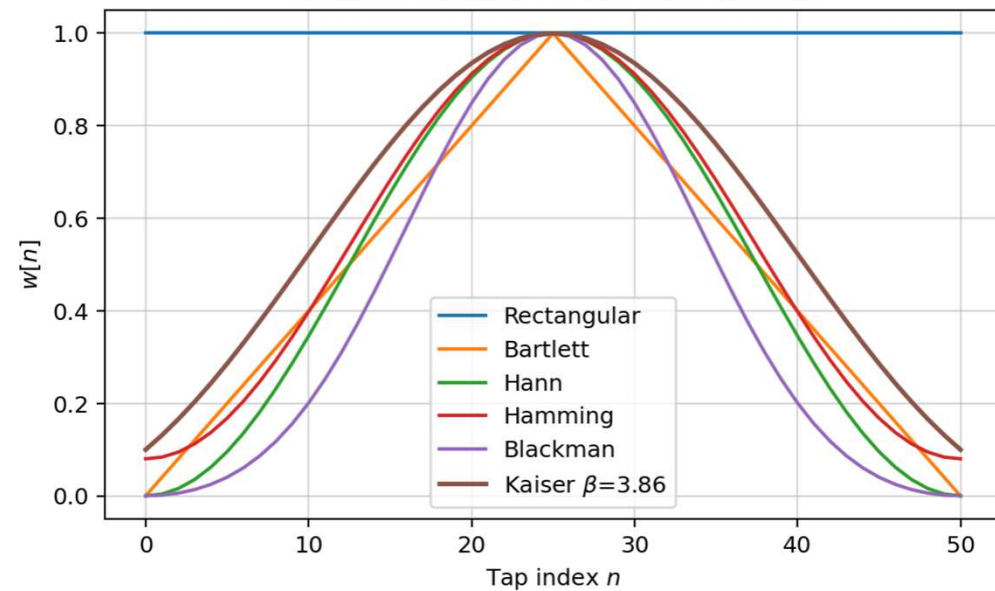


Normalized DTFTs of Common Window Functions

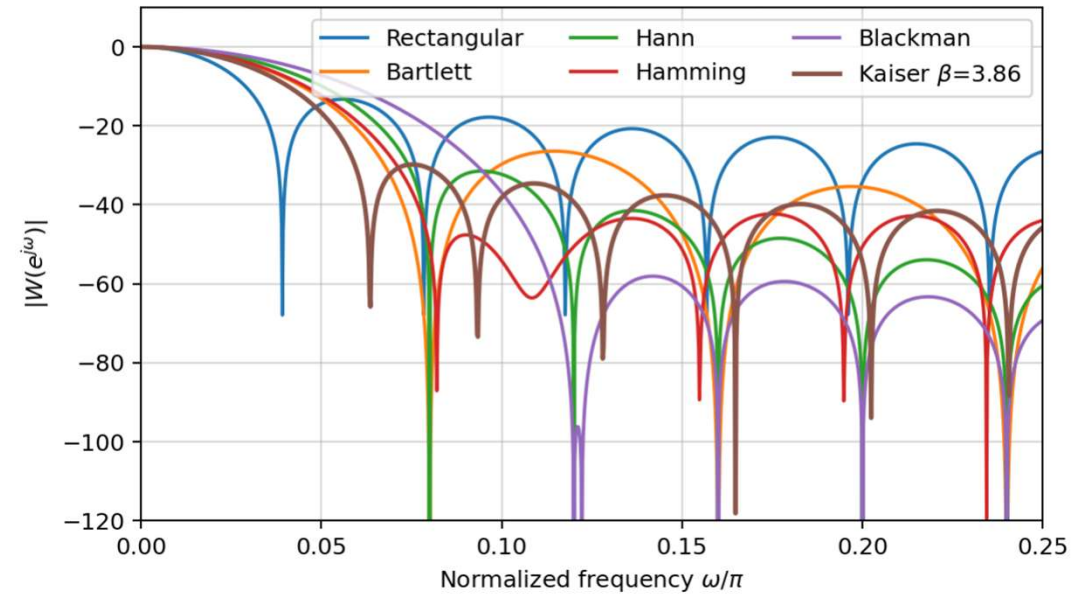


Window Functions in Time and Frequency, N=51

Common Windows in the Time Domain



Normalized DTFTs of Common Window Functions



Main-lobe width versus sidelobe attenuation

DTFTs of window functions

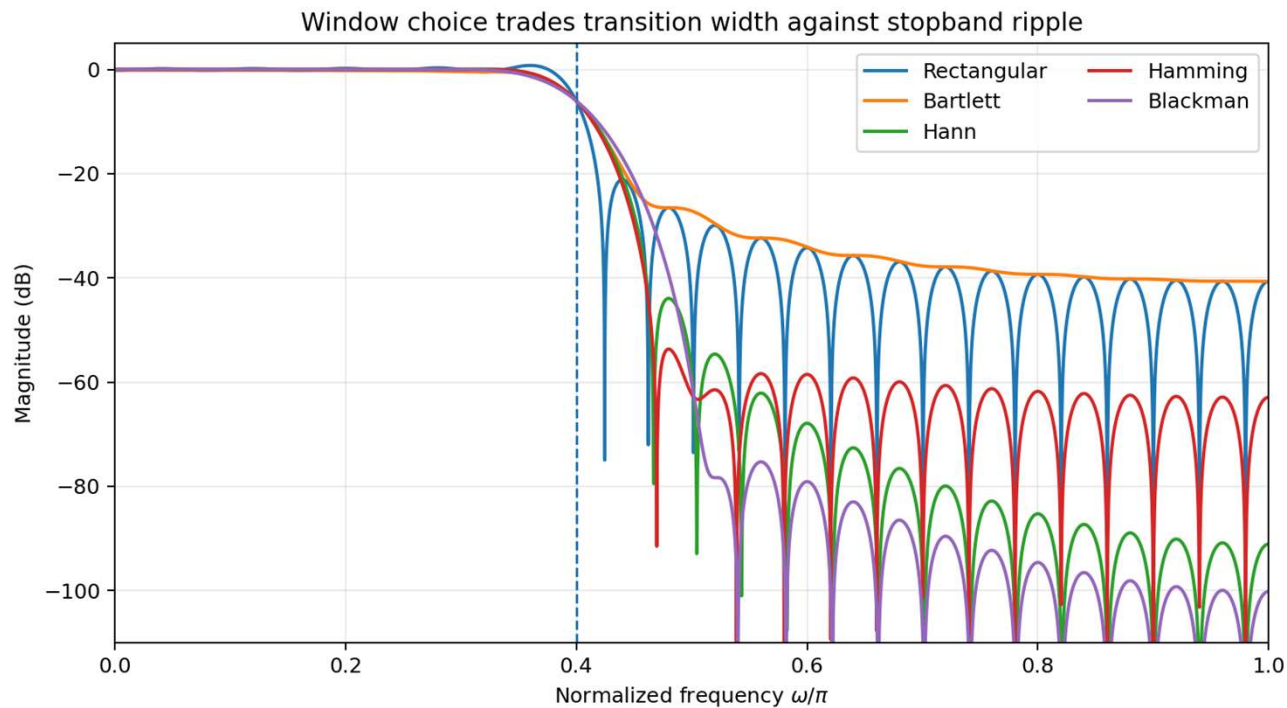
Window	Approx. first sidelobe	Approx. main-lobe width*	Use when...
Rectangular	≈ -13 dB	$\approx 4\pi/N$	Narrow transition dominates
Bartlett	≈ -26 dB	$\approx 8\pi/N$	Simple triangular taper
Hann	≈ -31 dB	$\approx 8\pi/N$	Balanced spectral leakage
Hamming	≈ -43 dB	$\approx 8\pi/N$	Better first sidelobe suppression
Blackman	≈ -58 dB	$\approx 12\pi/N$	Strong stopband suppression

*Width between the first nulls; approximate asymptotic rules for equal window length.

Good answer

There is no universally “best” window. The specification decides how much transition width can be traded for stopband suppression.

Same length and cutoff; different transition/ripple behavior



Rectangular

Sharpest transition, worst sidelobes.

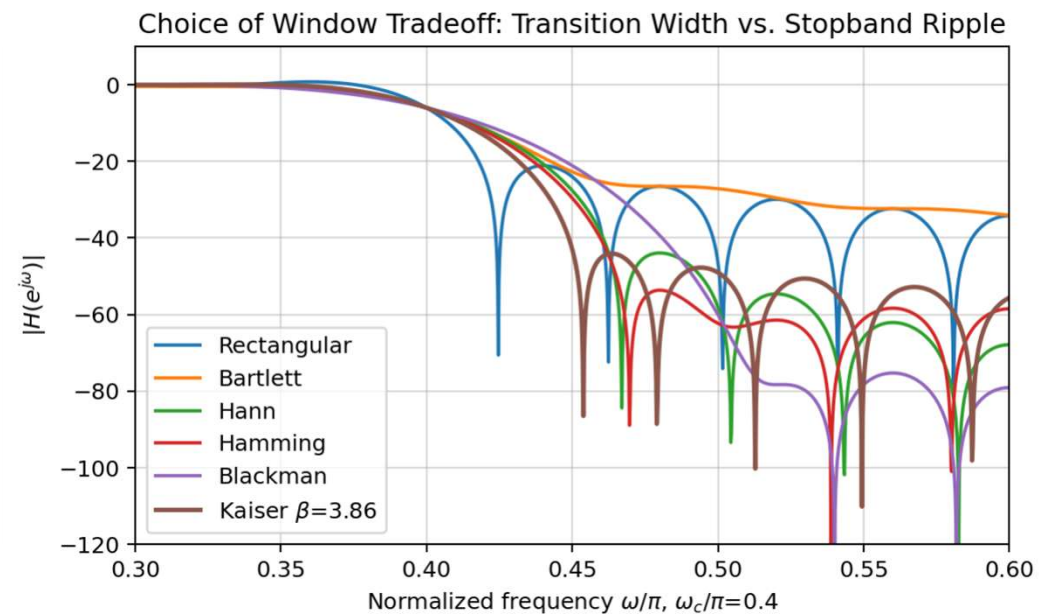
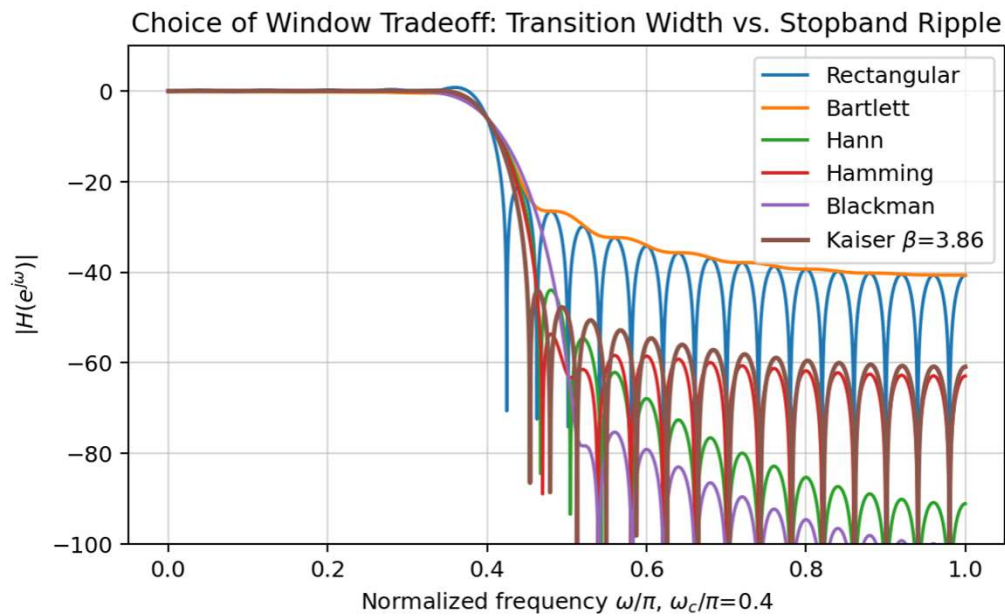
Hamming

A common compromise: wider transition, much lower first sidelobe.

Blackman

Strongest attenuation here, but broadest transition.

Truncated and Windowed Ideal LPF, $N=51$, $\omega_c = 0.4\pi$



A practical 51-tap Hamming lowpass

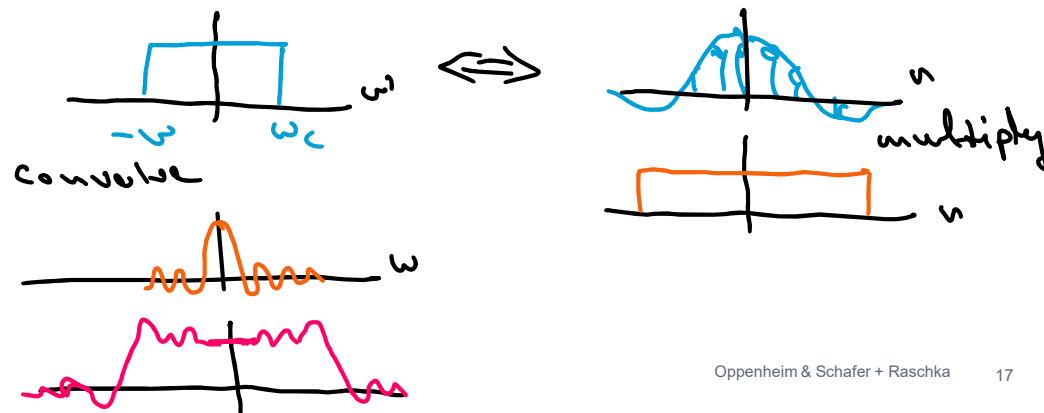
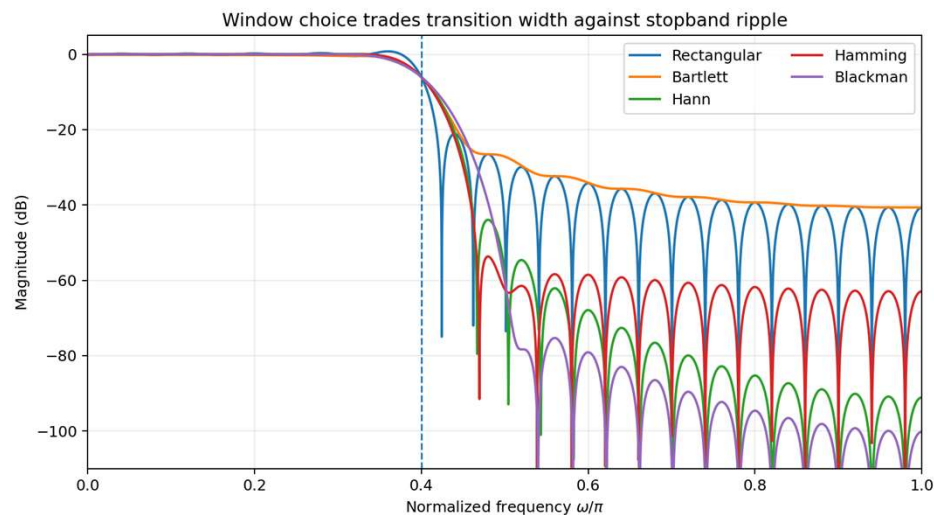
Specification sketch

$$N = 51, \quad \omega_c = 0.4\pi$$

$$h[n] = h_d[n]w_{\text{Ham}}[n]$$

Why Hamming?

We accept a somewhat wider transition in exchange for much better sidelobe rejection than rectangular truncation.



Longer FIRs buy a narrower transition—at a cost

$$\Delta\omega \propto \frac{1}{N}$$

Increase N

More taps generally narrow the transition for the same window family.

But pay for it

More multiplies, more memory, and larger linear-phase delay

$$\tau_g = \frac{N-1}{2}$$

ML viewpoint

A longer convolution kernel also means a larger temporal receptive field and more learned parameters if the kernel is trainable.

CONCEPT CHECK

Choose the design knob that matches the problem

A • Prompt

Stopband leakage is unacceptable, but transition width may be wider.

B • Prompt

Transition must be very narrow, but moderate ripple is acceptable.

C • Prompt

The filtered waveform must maintain relative timing across the passband.

CONCEPT CHECK

Choose the design knob that matches the problem

A • Prompt

Stopband leakage is unacceptable, but transition width may be wider.

Good answer

Choose a stronger taper such as Blackman or increase length.

B • Prompt

Transition must be very narrow, but moderate ripple is acceptable.

Good answer

Rectangular can give a narrower main lobe for the same N ; increasing N is another option.

C • Prompt

The filtered waveform must maintain relative timing across the passband.

Good answer

Use a linear-phase FIR or another phase-preserving strategy.

A 1-D convolution is a sliding dot product

$$y[n] = \sum_{k=0}^M h[k]x[n-k]$$

$$\mathbf{y} = \mathbf{T}(\mathbf{x})\mathbf{h}$$

convolution vs correlation
/ \
DSP ML
 PyTorch

DSP view

The impulse response $h[k]$ is designed to satisfy magnitude/phase specifications.

ML view

The same local dot product becomes a layer. The kernel may be initialized from DSP knowledge or learned from data.

$$y[n] = \sum_{k=0}^M h[k] \cdot x[n-k]$$

convolution with impulse response of length $M+1$

$$y[n] = \sum_{l=-M}^0 h[-l] x[n+l] = \sum_{k=0}^M \underbrace{h[M-k]}_{\triangleq w[k] = h[M-k]} x[n+k-M]$$

$$= \sum_{k=0}^M \underbrace{w[k]}_{\text{kernel of weights}} x[k+(n-M)]$$

input $x[k]$ shifted left by $n-M$

convolution can be implemented as matrix-vector mult.

$$\begin{matrix} \text{matrix} \\ \begin{bmatrix} 0 & 0 & x[0] \\ 0 & x[0] & x[1] \\ x[0] & x[1] & x[2] \end{bmatrix} \end{matrix} \cdot \begin{bmatrix} w[0] \\ w[1] \\ w[2] \\ \vdots \\ w[M] \end{bmatrix}$$

Conv1d uses cross-correlation ordering

$$y_{\text{DSP}}[n] = \sum_k h[k]x[n - k]$$

$$y_{\text{torch}}[n] = \sum_k w[k]x[n+k]$$

(Local-window correlation)

To reproduce textbook causal FIR

Left-pad by M samples and reverse the coefficient order before calling Conv1d.

$$w[k] = h[M - k]$$

Good answer

The operation is not “wrong convolution”; deep-learning libraries use a correlation convention because the kernel is usually learned anyway.

Fixed FIR taps inside PyTorch

```
h = torch.tensor(h_np, dtype=torch.float32)
x = torch.tensor(x_np).view(1, 1, -1)

# causal textbook convolution
kernel = h.flip(0).view(1, 1, -1)
x_pad = F.pad(x, (len(h)-1, 0))
y = F.conv1d(x_pad, kernel)
```

Shape

Input is (N,C,L). A single mono signal is (1,1,L).

Fixed weights

Use a tensor or freeze a Conv1d layer so the taps do not receive gradient updates.

Learnable weights

Leave `requires_grad=True` and the same coefficients become trainable model parameters.

Batch, channel, and length carry physical meaning

$$\mathbf{x} \in \mathbb{R}^{N \times C \times L}$$

N • Batch

Independent clips/examples processed together.

C • Channel

Mono, stereo, microphones, sensors, or learned feature maps.

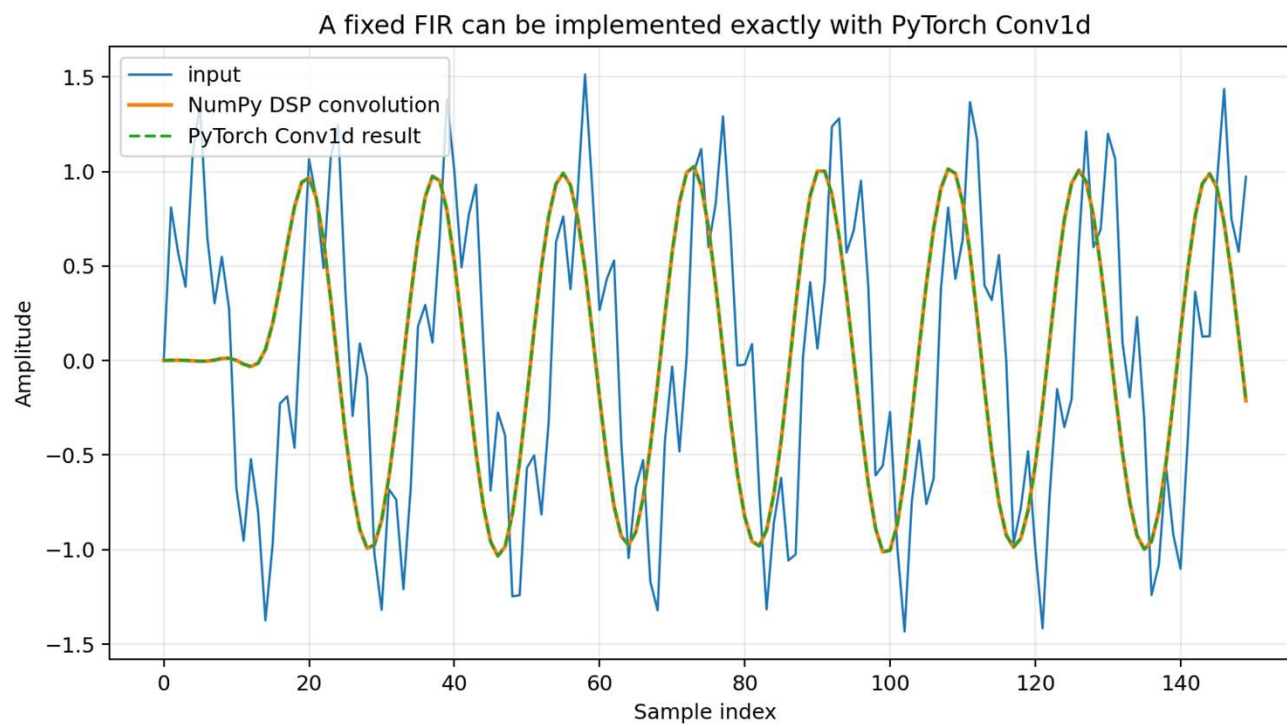
L • Length

Discrete-time sample index / temporal extent.

Example: 32 stereo one-second clips at 16 kHz $\rightarrow$ (32, 2, 16000)

DEMONSTRATION

NumPy convolution and PyTorch Conv1d agree



Numerical check

The supplied script reports a maximum mismatch on the order of 10^{-7} in float32.

Teaching point

Same filter, same signal-processing operation. The distinction is whether the coefficients are fixed by design or optimized from data.

Designed taps versus learned taps

Classical FIR design

Choose desired $H(e^{j\omega}) \rightarrow$ derive ideal $h[n] \rightarrow$ truncate/window $\rightarrow$ verify specifications.

Hybrid approach

Initialize Conv1d with a known bandpass/lowpass/filterbank and optionally fine-tune it.

Fully learned convolution

Choose kernel size and channels; backpropagation learns coefficients that reduce task loss.

The arithmetic is the same local weighted sum; the source of the weights is what changes.

DSP: specifications $\Rightarrow h[k]$ ML: data

CONTEXT

Translate the classical vocabulary into ML vocabulary

DSP concept	Mathematical role	ML connection
FIR filter	feedforward delay line	Conv1d with fixed or learnable kernel
Tap $h[k]$	impulse-response coefficient	kernel / weight
Window	controls truncation spectrum	inductive constraint on kernel support; not the same as ML regularization
Linear phase	constant group delay	preserves relative temporal alignment
Filter length	transition width + delay	receptive field + parameter count

Nuance: a classical window is a deterministic spectral-design device; calling it “kernel regularization” is only an analogy.

Lecture 7 synthesis

Linear phase

Symmetric FIR taps can provide constant group delay.

Ideal $\rightarrow$ practical

The brick-wall LPF becomes a sinc; finite implementation requires truncation and delay.

Window tradeoff

Lower sidelobes cost transition width.

PyTorch bridge

Conv1d performs the same local weighted sum; its weights may be fixed or learned.

Minute paper: What does a neural 1-D convolution add to the FIR operation itself?

Lecture 7 synthesis

Linear phase

Symmetric FIR taps can provide constant group delay.

Ideal $\rightarrow$ practical

The brick-wall LPF becomes a sinc; finite implementation requires truncation and delay.

Window tradeoff

Lower sidelobes cost transition width.

PyTorch bridge

Conv1d performs the same local weighted sum; its weights may be fixed or learned.

Minute paper: What does a neural 1-D convolution add to the FIR operation itself?

Strong answer

It does not change the local multiply-and-sum operation. It places that operation inside a trainable model so the kernel coefficients can be learned from a loss.